1. Introduction

Data fusion is a critical [**?**] process in Digital Twins, where multiple data sources from various applications need to be integrated to improve decision-making and extract valuable insights. Digital Twins represent virtual replicas of

Q1:

Listing 1: Query for q1_vehicle_count_observations

```
PREFIX traffic: <http://example.org/traffic#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema
    #>
SELECT ?location (COUNT(?vehicle) AS ?
    vehicleCount)
WHERE {
    ?vehicle a traffic:Vehicle ;
        traffic:atLocation ?location .
}
GROUP BY ?location
```

2. Query q2$_{situation_refinement_over_graph_simple}$

This query represents a straightforward situation refinement process in a

traffic knowledge graph. It aggregates vehicle count observations by location using the SUM function on the traffic:hasVehicleCount values, filtered to only include observations of type "VehicleCount". A HAVING clause applies a threshold (e.g., greater than 100) to detect high-traffic conditions, emitting triples for "TrafficJam" situations when the sum exceeds the limit. This simple structure is efficient for basic aggregation tasks and avoids nested queries, making it suitable for resource-constrained environments or initial prototyping in semantic web applications focused on real-time traffic analysis.

Listing 2: Query for q2_situation_refinement_over_graph_simple

```
PREFIX traffic: <http://example.org/traffic#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema
    #>
SELECT ?location (SUM(xsd:integer(?vc)) AS ?
    total)
WHERE {
    ?obs a traffic:Observation ;
        traffic:hasObservationType "
            VehicleCount" ;
        traffic:atLocation ?location ;
        traffic:hasVehicleCount ?vc .
}
GROUP BY ?location
HAVING (SUM(xsd:integer(?vc)) > 100)
```

3. Query q2$_{situation_refinement_over_graph_medium}$

Building on the simple version, this intermediate SPARQL query introduces a subquery for pre-aggregation of vehicle counts per location. It first computes the sum of vehicle counts within the inner SELECT, applying a FILTER to ensure the observation type matches "VehicleCount", then groups by location and filters totals exceeding the threshold via a HAVING clause in the outer scope. This nested approach enhances modularity, allowing for easier extension (e.g., adding more filters) while maintaining clarity. It is ideal for scenarios requiring layered reasoning in ontologies, such as refining traffic situations from aggregated observations in intelligent transportation systems.

Listing 3: Query for q2_situation_refinement_over_graph_medium

```
PREFIX traffic: <http://example.org/
    traffic#>
PREFIX xsd: <http://www.w3.org/2001/
    XMLSchema#>
SELECT ?location ?total
WHERE {
    SELECT ?location (SUM(xsd:integer(?vc
        )) AS ?total)
    WHERE {
        ?obs a traffic:Observation ;
            traffic:hasObservationType ?
                type ;
            traffic:atLocation ?location
                ;
            traffic:hasVehicleCount ?vc .
        FILTER(?type = "VehicleCount")
    }
    GROUP BY ?location
    HAVING (?total > 100)
}
```

4. Query q2$_{situation_refinement_over_graph_complex}$

This advanced SPARQL query employs a multi-layered structure for robust situation refinement, incorporating VALUES to restrict observation types, FILTER EXISTS to validate entity shapes (e.g., ensuring locations are properly linked), and nested subqueries for computing both SUM and AVG aggregates on vehicle counts. The innermost subquery casts counts to integers and applies filters, while the outer layers group by location and enforce the threshold via HAVING on the sum. Although it includes an unused average for illustration, this complexity supports sophisticated validation and multi-metric analysis, making it suitable for high-fidelity applications in semantic reasoning, such as detecting traffic jams with additional quality checks in dynamic urban ontologies.

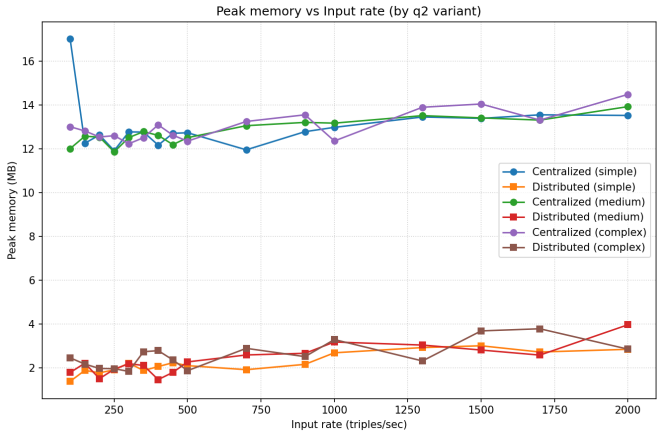

Figure 1: The semantic JDL model [**?**]

Listing 4: Query for q2_situation_refinement_over_graph_complex

```
PREFIX traffic: <http://example.org/traffic#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema
    #>
SELECT ?location ?total
WHERE {
  SELECT ?location (SUM(?vc_int) AS ?total) (
      AVG(?vc_int) AS ?avg)
  WHERE {
    {
      SELECT ?obs ?location (xsd:integer(?vc)
          AS ?vc_int)
      WHERE {
        VALUES ?otype {"VehicleCount"}
        ?obs a traffic:Observation ;
             traffic:hasObservationType ?
                 otype ;
             traffic:atLocation ?location ;
             traffic:hasVehicleCount ?vc .
        FILTER EXISTS { ?obs traffic:
            atLocation ?loc2 }
      }
    }
  }
  GROUP BY ?location
  HAVING (SUM(?vc_int) > 100)
}
```

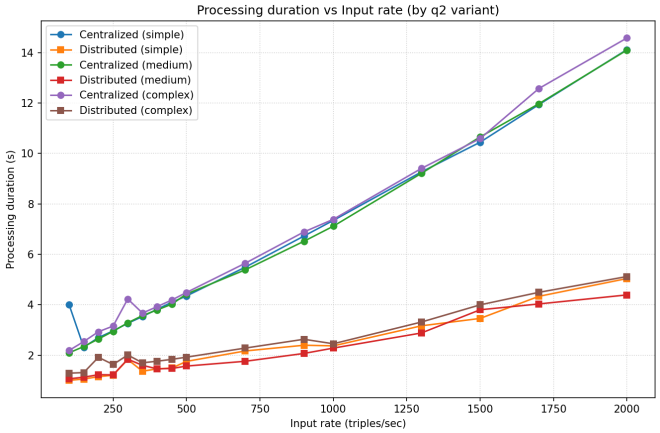

Figure 2: The semantic JDL model [**?**]

5. Results

References